

Experiment 2

Aim: Create a Blockchain using Python Hash

Theory:

Blockchain is a distributed and immutable ledger that records data in a sequence of linked blocks. Each block contains a timestamp, proof (from Proof-of-Work), and the hash of the previous block, ensuring integrity and security.

In this program:

1. **Block Creation** – A block is generated using `create_block()` with a proof and previous hash.
2. **Proof-of-Work (PoW)** – Implemented to mine a block by solving a cryptographic puzzle (finding a hash with leading zeros).
3. **Hashing** – SHA-256 algorithm ensures data immutability.
4. **Validation** – `is_chain_valid()` checks that every block correctly references the previous hash and satisfies PoW conditions.
5. **Flask Web API** – Routes `/mine_block`, `/get_chain`, and `/is_valid` allow mining, fetching the chain, and verifying blockchain integrity through a browser or API client.

This program runs on a local server and simulates mining a blockchain without a network of nodes, making it an ideal introductory model to understand blockchain basics.

Code:

1. Importing Required Libraries

- **datetime** → to timestamp each block
- **hashlib** → to generate SHA-256 hashes

- **json** → to encode blocks as JSON for hashing
- **Flask** → to run the blockchain API server

2. Blockchain Class – Core Logic

This contains everything needed to:

- create blocks
- store the chain
- implement Proof of Work
- validate chain integrity

3. Flask Web API

Provides endpoints:

- `/mine_block` → mine a new block
- `/get_chain` → fetch full blockchain
- `/is_valid` → verify blockchain

Program and Output:

```
from flask import Flask, jsonify,
request
import datetime
import hashlib
import json
class Blockchain:

    def __init__(self):
        # Stores all blocks
        self.chain = []

        # Stores transactions waiting
to be mined
        self.pending_transactions =
[]

        # Mining difficulty: hash
must begin with 000
        self.difficulty = 3
```

```
# Create the Genesis Block
self.create_block(
    proof=1,
    previous_hash="0",
    transactions=[],
    miner="Genesis"
)

#
-----
# Create a new block
#
-----
def create_block(self, proof,
previous_hash,
```

```

        return self.chain[-1]

transactions=None,
miner="Student-Miner"):

    if transactions is None:
        transactions = []

    block = {
        "index": len(self.chain)
+ 1,
        "timestamp":
str(datetime.datetime.now()),
        "transactions":
transactions,
        "transaction_count":
len(transactions),
        "proof": proof,
        "nonce": proof,
        "difficulty":
self.difficulty,
        "miner": miner,
        "previous_hash":
previous_hash
    }

    # Generate the hash of the
complete block
    block["block_hash"] =
self.hash(block)

    # Add block to blockchain
self.chain.append(block)

    return block

#
-----
-----

    # Get the most recently added
block
#
-----
-----

    def get_previous_block(self):

        return self.chain[-1]

#
-----
-----

    # Add a new transaction
#
-----
-----

    def create_transactions(self,
sender, receiver, amount):

        transaction = {
            "transaction_id":
hashlib.sha256(
f"{sender}{receiver}{amount}{datetime
.datetime.now()}".encode()
).hexdigest()[:12],
            "sender": sender,
            "receiver": receiver,
            "amount": amount,
            "timestamp":
str(datetime.datetime.now())
        }

        # Add transaction to pending
pool

self.pending_transactions.append(tran
saction)

        # Return the block number in
which it will be stored
        return
self.get_previous_block()["index"] +
1

#
-----
-----

    # Proof of Work

```

```

#
-----

def proof_of_work(self,
previous_proof):

    nonce = 1

    while True:

        calculation = nonce ** 2
- previous_proof ** 2

        hash_result =
hashlib.sha256(
str(calculation).encode()
        ).hexdigest()

        # Golden nonce condition
        if
hash_result.startswith("000"):
            return nonce

        nonce += 1

#
-----
-----

# Generate SHA-256 hash for a
block
#
-----
-----

def hash(self, block):

    # Do not include block_hash
while calculating the hash
        block_copy = block.copy()
        block_copy.pop("block_hash",
None)

        encoded_block = json.dumps(
            block_copy,

```

```

        sort_keys=True
        ).encode()

        return
hashlib.sha256(encoded_block).hexdigest()

#
-----
-----

# Validate the complete
blockchain
#
-----
-----

def is_chain_valid(self, chain):

    if not chain:
        return False

    for position in range(1,
len(chain)):

        previous_block =
chain[position - 1]
        current_block =
chain[position]

        # 1. Check previous block
connection
        expected_previous_hash =
self.hash(previous_block)

        if
current_block["previous_hash"] !=
expected_previous_hash:
            return False

        # 2. Verify Proof of Work
        previous_proof =
previous_block["proof"]
        current_proof =
current_block["proof"]

```

```

        calculation = (
            current_proof ** 2 -
            previous_proof ** 2
        )

        hash_result =
hashlib.sha256(

str(calculation).encode()
        ).hexdigest()

        if not
hash_result.startswith("000"):
            return False

        # 3. Verify the stored
block hash
            calculated_block_hash =
self.hash(current_block)

            if
current_block["block_hash"] !=
calculated_block_hash:
                return False

            # 4. Verify transaction
count
                if
current_block["transaction_count"] !=
len(

current_block["transactions"]
                ):
                    return False

            return True

# Creating a Blockchain
blockchain = Blockchain()
#
-----
-----
# Mine a new block

```

```

#
-----
-----
def
mine_block(miner_name="Student-Miner"
):

        # Check for transactions waiting
to be mined
        if not
blockchain.pending_transactions:
            return {
                "status":
"NO_TRANSACTION",
                "message": "Mining
cancelled. There are no pending
transactions."
            }

        # Take a snapshot of the pending
transactions
            transactions_to_mine =
blockchain.pending_transactions.copy(
)

            # Remove them from the pending
pool before mining

            blockchain.pending_transactions.clear
()

            # Get the latest block
            previous_block =
blockchain.get_previous_block()

            # Get the proof from the previous
block
                previous_proof =
previous_block["proof"]

            # Solve the Proof-of-Work puzzle
                golden_nonce =
blockchain.proof_of_work(previous_pro
of)

```

```

        # Get the hash of the previous
        block
        previous_hash =
        blockchain.hash(previous_block)

        # Create the new block
        mined_block =
        blockchain.create_block(
            proof=golden_nonce,
            previous_hash=previous_hash,

            transactions=transactions_to_mine,
            miner=miner_name
        )

        # Return useful mining
        information
        return {
            "status": "SUCCESS",
            "message": "New block mined
            successfully!",
            "block_number":
            mined_block["index"],
            "miner":
            mined_block["miner"],
            "transaction_count":
            mined_block["transaction_count"],
            "transactions":
            mined_block["transactions"],
            "golden_nonce":
            mined_block["nonce"],
            "difficulty":
            mined_block["difficulty"],
            "previous_hash":
            mined_block["previous_hash"],
            "block_hash":
            mined_block["block_hash"],
            "timestamp":
            mined_block["timestamp"]
        }
    #
    -----
    -----

```

```

    # Display the complete blockchain
    #
    -----
    -----
    def get_chain():

        formatted_blocks = []

        for block in blockchain.chain:

            formatted_blocks.append({
                "block_number":
                block["index"],
                "created_at":
                block["timestamp"],
                "miner": block["miner"],
                "transaction_count":
                block["transaction_count"],
                "transactions":
                block["transactions"],
                "nonce": block["nonce"],
                "difficulty":
                block["difficulty"],
                "previous_hash":
                block["previous_hash"],
                "block_hash":
                block["block_hash"]
            })

        return {
            "status": "SUCCESS",
            "message": "Blockchain
            retrieved successfully.",
            "total_blocks":
            len(formatted_blocks),
            "blocks": formatted_blocks
        }

    # Verify the integrity of the
    blockchain
    def check_blockchain():
        valid =
        blockchain.is_chain_valid(blockchain.
        chain)

```

```

        if valid:
            return {
                'status': 'VALID',
                'message': 'Blockchain
verification successful.',
                'total_blocks':
len(blockchain.chain)
            }

        return {
            'status': 'INVALID',
            'message': 'Blockchain
verification failed. The chain has
been modified.',
            'total_blocks':
len(blockchain.chain)
        }
#
-----
-----
# Verify blockchain integrity
#
-----
-----
def check_blockchain():

    total_blocks =
len(blockchain.chain)
    valid =
blockchain.is_chain_valid(blockchain.
chain)

    if valid:

        return {
            "status": "VALID",
            "message": "Blockchain
integrity verified successfully.",
            "total_blocks":
total_blocks,
            "checks": {

"previous_hash_links": "PASSED",

```

```

            "proof_of_work":
"PASSED",
            "block_hashes":
"PASSED",
            "transaction_counts":
"PASSED"
        }
    }

    return {
        "status": "INVALID",
        "message": "Blockchain
verification failed.",
        "total_blocks": total_blocks,
        "checks": {
            "previous_hash_links":
"FAILED or MODIFIED",
            "proof_of_work": "FAILED
or MODIFIED",
            "block_hashes": "FAILED
or MODIFIED",
            "transaction_counts":
"FAILED or MODIFIED"
        }
    }

    while True:

        print("\n" + "=" * 55)
        print("                PYCHAIN
BLOCKCHAIN")
        print("=" * 55)
        print("1. Add Transaction")
        print("2. Mine Pending
Transactions")
        print("3. View Blockchain")
        print("4. Verify Blockchain")
        print("5. Exit")
        print("-" * 55)

        choice = input("Enter your
choice: ")

        if choice == "1":

```

```

        print("\n--- New Transaction
---")

        sender = input("Sender : ")
        receiver = input("Receiver : ")

    try:
        amount =
float(input("Amount : "))

        if amount <= 0:
            print("Amount must be
greater than zero.")
            continue

        except ValueError:
            print("Invalid amount.
Please enter a number.")
            continue

        block_number =
blockchain.create_transactions(
            sender,
            receiver,
            amount
        )

        print("\nTransaction
accepted!")
        print(f"Sender :
{sender}")
        print(f"Receiver :
{receiver}")
        print(f"Amount :
{amount}")
        print(f"Target Block :
#{block_number}")

        elif choice == "2":

            miner_name = input("\nEnter
miner name: ")

```

```

        result =
mine_block(miner_name)

        print("\n--- Mining Result
---")

        if result["status"] ==
"SUCCESS":
            print("Block mined
successfully!")
            print(f"Block Number :
#{result['block_number']}")
            print(f"Miner :
{result['miner']}")
            print(f"Transactions :
{result['transaction_count']}")
            print(f"Golden Nonce :
{result['golden_nonce']}")
            print(f"Difficulty :
{result['difficulty']}")
            print(f"Previous Hash :
{result['previous_hash']}")
            print(f"Block Hash :
{result['block_hash']}")
            print(f"Timestamp :
{result['timestamp']}")

            else:
                print(result["message"])

        elif choice == "3":

            result = get_chain()

            print("\n" + "=" * 55)
            print("
BLOCKCHAIN LEDGER")
            print("=" * 55)
            print(f"Total Blocks:
{result['total_blocks']}")

            for block in
result["blocks"]:

```

```

        print("\n" + "-" * 55)
        print(f"BLOCK
#{block['block_number']}")
        print("-" * 55)
        print(f"Created      :
{block['created_at']}")
        print(f"Miner       :
{block['miner']}")
        print(f"Transactions :
{block['transaction_count']}")
        print(f"Nonce       :
{block['nonce']}")
        print(f"Difficulty  :
{block['difficulty']}")
        print(f"Previous Hash :
{block['previous_hash']}")
        print(f"Block Hash   :
{block['block_hash']}")

        if block["transactions"]:
            print("\nTransaction
Details:")

            for transaction in
block["transactions"]:
                print(
                    f"
{transaction['sender']} -> "

                    f"{transaction['receiver']} | "
                    f"Amount:
{transaction['amount']} | "
                    f"ID:
{transaction['transaction_id']}")
            else:

```

```

        print("\nNo
transactions in this block.")

```

```

=====
PYCHAIN BLOCKCHAIN
=====
1. Add Transaction
2. Mine Pending Transactions
3. View Blockchain
4. Verify Blockchain
5. Exit
-----
Enter your choice: 1

--- New Transaction ---
Sender   : Saniya Kadam
Receiver : Viraj Kadam
Amount   : 10000

Transaction accepted!
Sender    : Saniya Kadam
Receiver  : Viraj Kadam
Amount    : 10000.0
Target Block : #2

```

```

        for check, status in
result["checks"].items():

        print(f"{check.replace('_', '
').title():25} : {status}")

        elif choice == "5":

            print("\nBlockchain
application closed.")
            break

        else:

            print("\nInvalid choice.
Please select 1 to 5.")

```

```
=====
PYCHAIN BLOCKCHAIN
=====
1. Add Transaction
2. Mine Pending Transactions
3. View Blockchain
4. Verify Blockchain
5. Exit
-----
Enter your choice: 1

--- New Transaction ---
Sender   : Om Manglani
Receiver : Aaryan Shetye
Amount   : 500

Transaction accepted!
Sender    : Om Manglani
Receiver  : Aaryan Shetye
Amount    : 500.0
Target Block : #2
```

```
=====
PYCHAIN BLOCKCHAIN
=====
1. Add Transaction
2. Mine Pending Transactions
3. View Blockchain
4. Verify Blockchain
5. Exit
-----
Enter your choice: 2

Enter miner name: Krish Dambir

--- Mining Result ---
Block mined successfully!
Block Number   : #2
Miner          : Krish Dambir
Transactions    : 2
Golden Nonce    : 533
Difficulty      : 3
Previous Hash   : d02fdea89f8e1b0bfc8982d40b78c3f3cbcd37338ca326ce352aa5d47230d97b
Block Hash      : 240d6888b563142842fc0d614b74f320335a48fd54f467d5eb956e5eab3c2e55
Timestamp       : 2026-08-26 06:43:39.647620
```

```
=====
PYCHAIN BLOCKCHAIN
=====
1. Add Transaction
2. Mine Pending Transactions
3. View Blockchain
4. Verify Blockchain
5. Exit
-----
Enter your choice: 3

=====
BLOCKCHAIN LEDGER
=====
Total Blocks: 2

-----
BLOCK #1
-----
Created      : 2026-08-26 06:41:19.619650
Miner        : Genesis
Transactions : 0
Nonce        : 1
Difficulty    : 3
Previous Hash : 0
Block Hash    : d02fdea89f8e1b0bfc8982d40b78c3f3cbcd37338ca326ce352aa5d47230d97b

No transactions in this block.
```

```
-----
BLOCK #2
-----
Created      : 2026-08-26 06:43:39.647620
Miner        : Krish Dambir
Transactions : 2
Nonce        : 533
Difficulty    : 3
Previous Hash : d02fdea89f8e1b0bfc8982d40b78c3f3cbcd37338ca326ce352aa5d47230d97b
Block Hash    : 240d6888b563142842fc0d614b74f320335a48fd54f467d5eb956e5eab3c2e55

Transaction Details:
  Saniya Kadam -> Viraj Kadam | Amount: 10000.0 | ID: 8f893f44df9e
  Om Manglani -> Aaryan Shetye | Amount: 500.0 | ID: 48eac1bb06fc

=====
PYCHAIN BLOCKCHAIN
=====
1. Add Transaction
2. Mine Pending Transactions
3. View Blockchain
4. Verify Blockchain
5. Exit
-----
Enter your choice: 4

--- Blockchain Verification ---
Status      : VALID
Message      : Blockchain integrity verified successfully.
Total Blocks : 2
```

```
Verification Checks:
Previous Hash Links      : PASSED
Proof Of Work           : PASSED
Block Hashes            : PASSED
Transaction Counts      : PASSED

=====
PYCHAIN BLOCKCHAIN
=====
1. Add Transaction
2. Mine Pending Transactions
3. View Blockchain
4. Verify Blockchain
5. Exit
-----
Enter your choice: 5

Blockchain application closed.
```

Conclusion:

We successfully created a basic blockchain using Python and Flask that supports block mining, chain retrieval, and validity checks. Proof-of-Work ensures security by making block creation computationally intensive, while cryptographic hashing guarantees immutability. The implementation demonstrates core blockchain principles—decentralization, immutability, and consensus—in a simplified environment, providing a strong foundation for building more advanced blockchain systems.